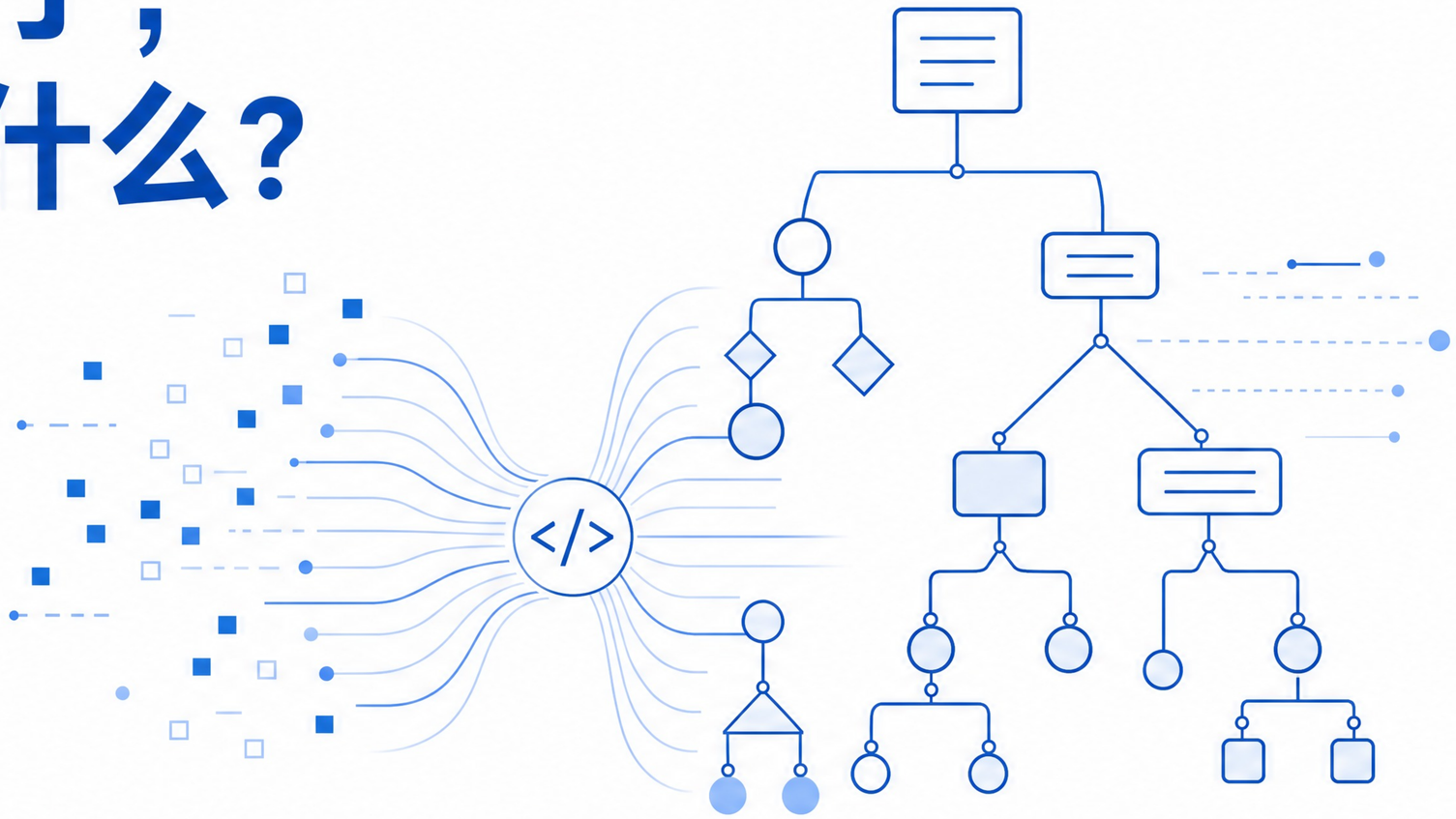
AI都能写代码了，我们还要学习什么？

从编译原理谈起



王言博士（0611）

在代码生成门槛降低的今天，
聊聊专业训练留下的系统性判断力

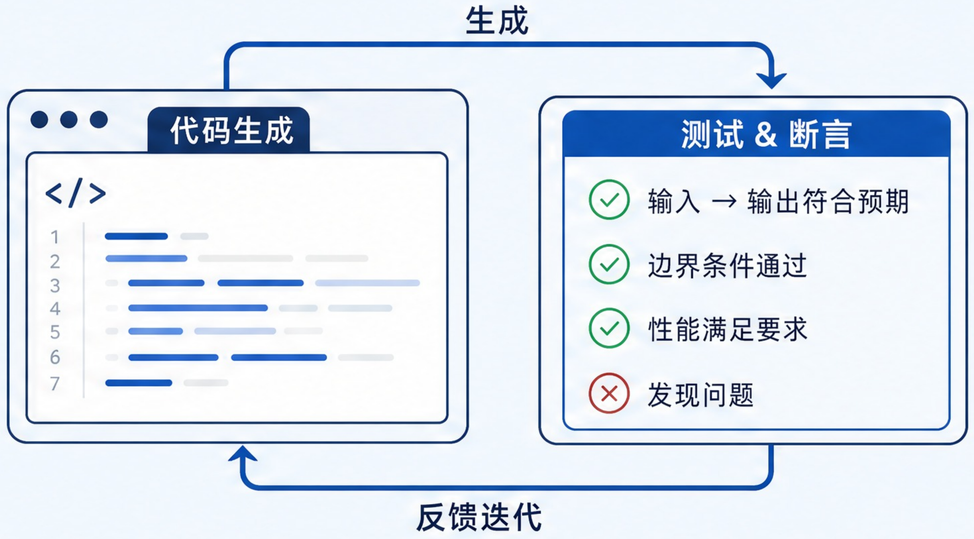
今天我们一起讨论的两个困惑

01 困惑一 · 方向之问

未来不做编译器或网管，学底层课有什么用？

02 困惑二 · 动力之问

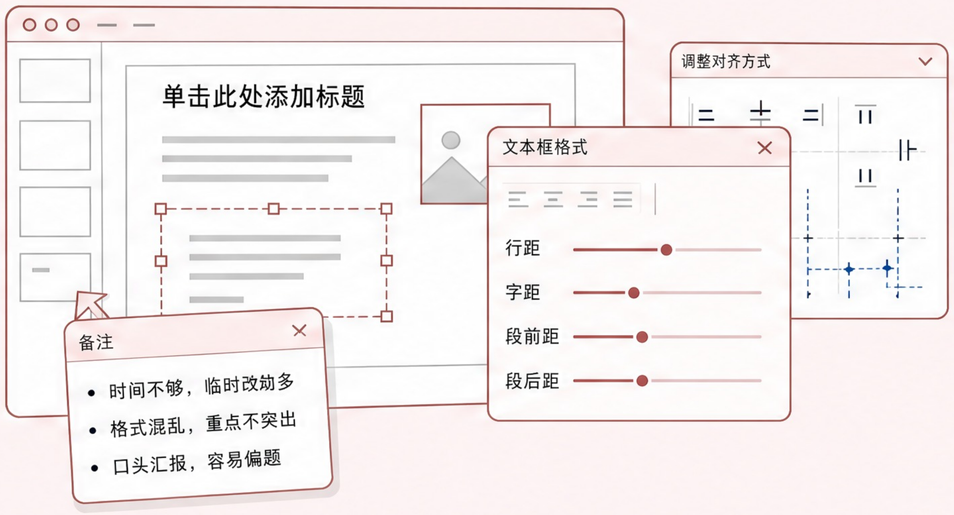
大模型几分钟写完大实验，为什么还要自己手写？



从一个小编译器谈起，厘清专业训练在 AI 时代的真实意义

故事从一次令人头疼的实习汇报开始

手动调整格式的日常困扰



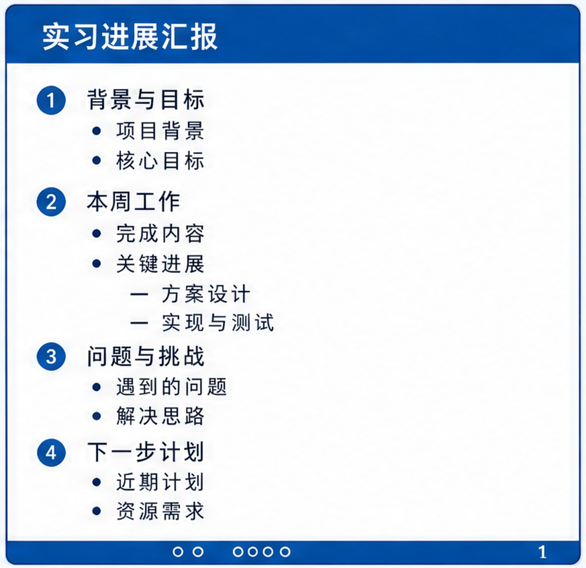
手动调格式费时，口头汇报易失焦

纯文本转换为演示文稿

纯文本大纲（按 Tab 缩进）

```
实习进展汇报
├── 背景与目标
│   ├── 项目背景
│   └── 核心目标
├── 本周工作
│   ├── 完成内容
│   ├── 关键进展
│   │   ├── 方案设计
│   │   └── 实现与测试
├── 问题与挑战
│   ├── 遇到的问题
│   └── 解决思路
└── 下一步计划
    ├── 近期计划
    └── 资源需求
```

生成的 LaTeX Beamer 幻灯片



思路清晰

层次分明，逻辑自然

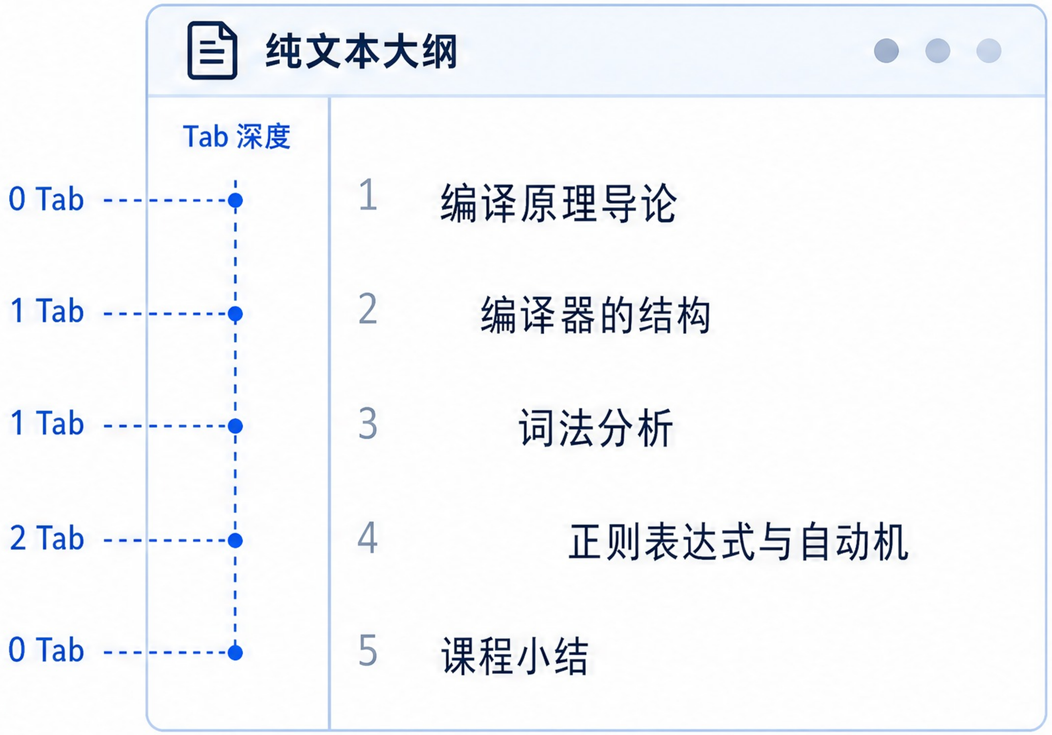
沟通高效

重点突出，交流顺畅

源码沉淀

每日记录，持续积累

一段随手写下的大纲，怎么就成了编译器能懂的语言？



形式化文法映射规则

顶格行（0 Tab）
→ **幻灯片标题**

1 个 Tab
→ **一级列表项**

2 个 Tab
→ **嵌套子列表项**

文法约定： 约定了明确规则，缩进文本就是严谨的形式化语言

规则映射： 行首顶格表示幻灯片标题，Tab 缩进深度严格对应列表嵌套层级

专业启发：我们可以创造一种符合习惯的 DSL，用自动机去处理它

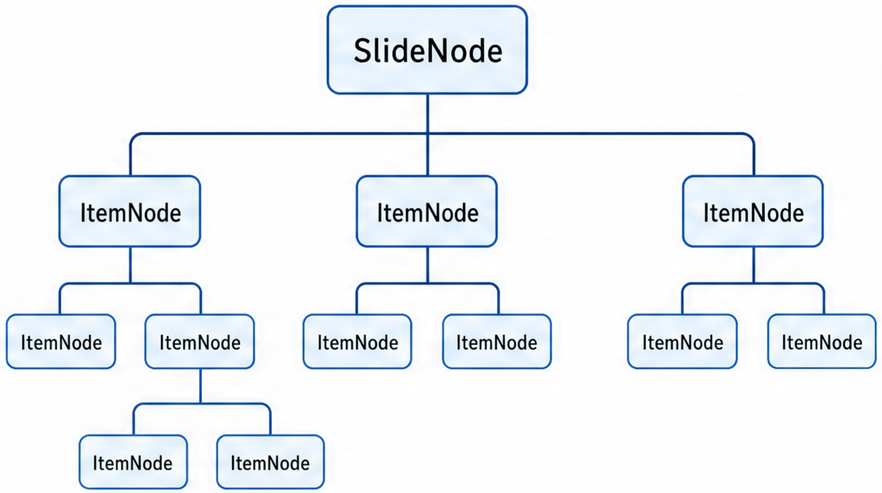
先把输入变成一棵抽象语法树

抽象语法树将解析输入与生成目标彻底解耦

纯文本输入（缩进大纲）

```
先把输入变成一棵抽象语法树
    为什么需要 AST
        解析与生成解耦
        结构化表达语义
    前端：解析输入
        扫描缩进深度
        递归构建节点树
    后端：生成输出
        遍历 AST 节点
        输出 Beamer 代码
```

AST 抽象语法树（解耦两端的核心枢纽）



LaTeX Beamer 目标代码

```latex
\begin{frame}{先把输入变成一棵抽象语法树}
  \begin{itemize}
    \item 为什么需要 AST
      \begin{itemize}
        \item 解析与生成解耦
        \item 结构化表达语义
      \end{itemize}
    \item 前端：解析输入
      \begin{itemize}
        \item 扫描缩进深度
        \item 递归构建节点树
      \end{itemize}
  \end{itemize}
\end{frame}
```

前端负责解析

扫描文本的 Tab 缩进深度，
递归构建节点嵌套树

解耦两端的核心枢纽

抽象语法树将解析输入与
生成目标彻底解耦

后端负责生成

遍历 AST 节点，
递归输出 LaTeX Beamer 目标代码
（\begin{frame} 与 \item）

在这个过程中编译原理有什么用?

1. 可行性判断

认出缩进大纲是一门形式化语言，
可以用程序系统化解决

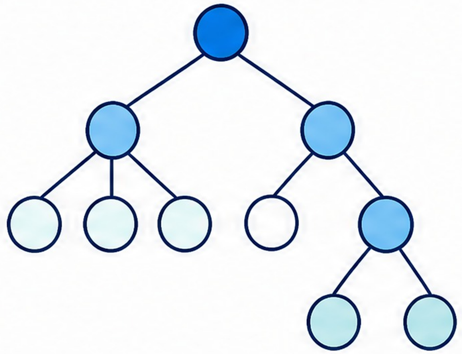
2. 架构解耦直觉

自然选择 AST 连接前后端，
避免脆弱的字符串逻辑

3. 难度边界意识

看清这只是简单的递归下降解析，
半天即可实现

AST（抽象语法树）



结构化表达，语义与格式分离

AST 100% 复用

Beamer 后端（默认）

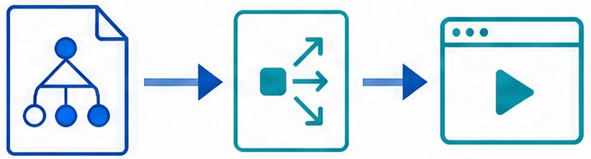
- 生成 LaTeX Beamer 源文件
- 现有流程完全兼容

AST 100% 复用

Reveal.js HTML 后端（新增）

- 生成 Reveal.js 离线 HTML 文件
- 前端解析不重写

从容应对变更： 如果明天要求在浏览器离线放映，AST 完全保留，只需新增一个 Reveal.js HTML 后端



不仅是做幻灯片：编译思维在真实工程里帮我们做什么？

抽象工程模式

将复杂业务收敛为领域专有语言与状态机

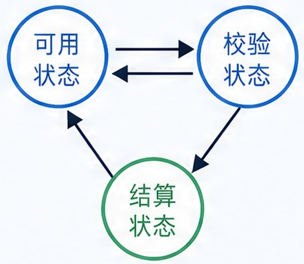
混乱的业务规则

满减 | 会员
品类 | 时间段
叠加 | 互斥
优先级 | ···

领域专有语言（DSL）

```
rule: 满减
when: 购物金额 >= 200
and: 会员等级 >= 银卡
then: 减免 30
priority: 10
```

状态机收敛（执行语义）



将转换任务拆解为多阶段流水线

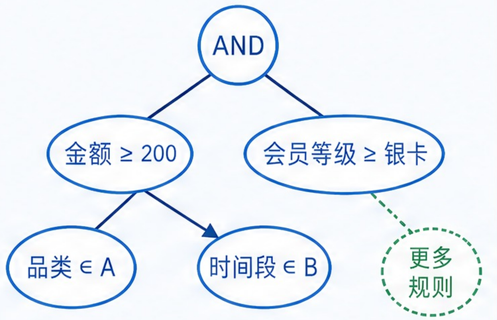
具体业务场景

1 电商满减折扣规则：从复杂分支到表达式树

嵌套数十层条件分支（脆弱难维护）

```
if (金额 >= 200) {
 if (会员等级 >= 银卡) {
  if (品类 in A) {
   if (时间段 _in B) {
    if ( ... ) {
      ...
    }
   }
  }
 }
}
...
```

规则表达式树（结构清晰可演化）



2 代码提交前：利用语法树进行静态安全检查

核心启发： 形式化抽象与分阶段转换，是解决复杂系统逻辑泥潭的通用武器

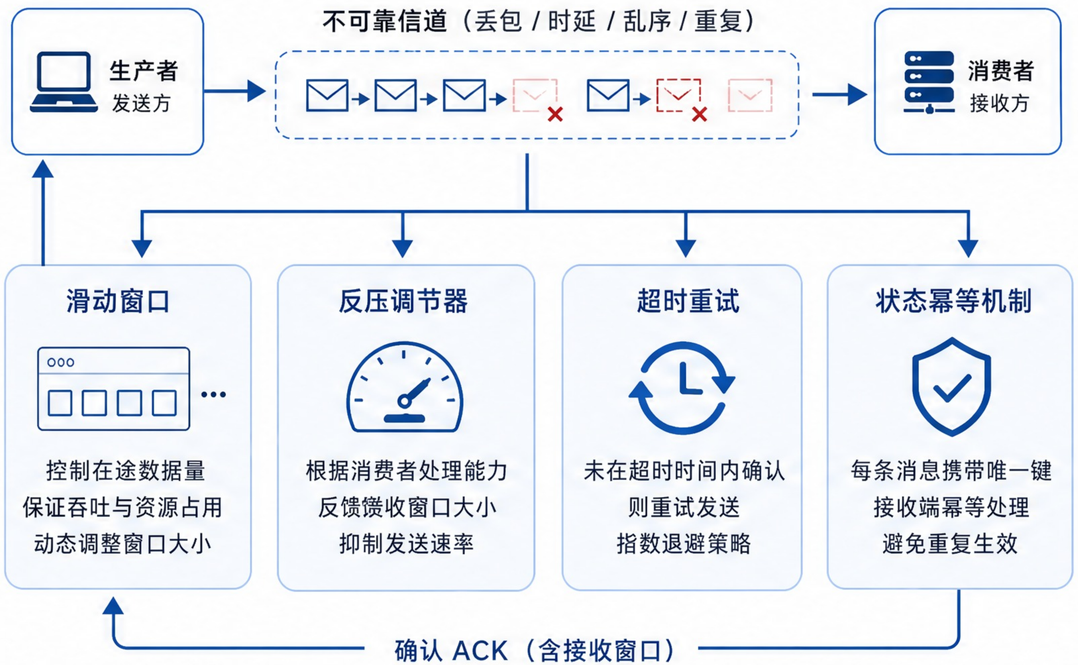
计算机网络：面对不可靠与容量错配的协作法则

抽象模式： 不可靠信道上的端到端一致性保证；异步生产消费中的容量错配与反压流控

真实案例： 地铁扫码支付遇到网络转菊花超时，通过唯一去重键实现状态幂等避免重复扣款；
搜索框飞速打字导致迟到请求覆盖新结果，通过流控防抖与乱序重排保证一致性

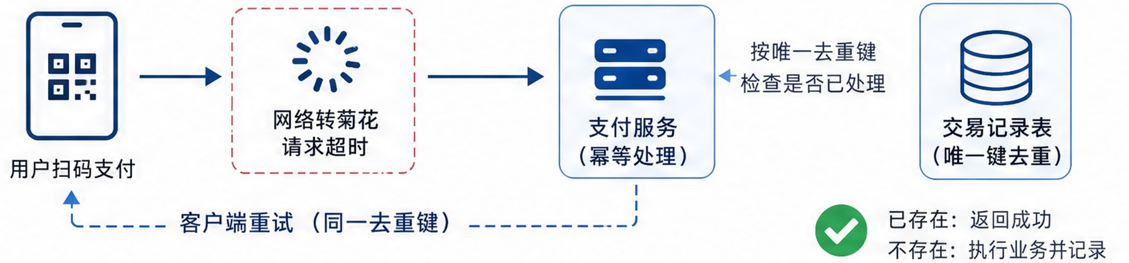
核心启发： 哪怕只写单机代码，任何涉及异步生产与消费的场景，都需要网络的容错与流控思维

抽象协作模型

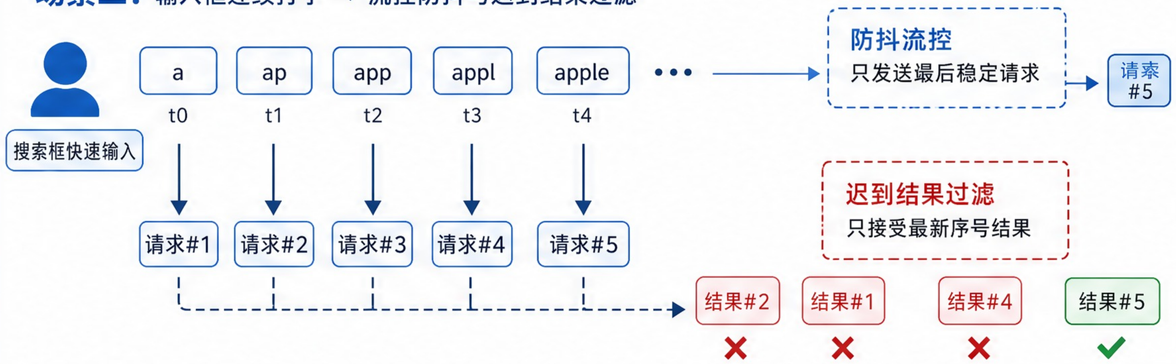


具体业务场景

场景一： 扫码支付网络超时 → 唯一去重键防重复扣款



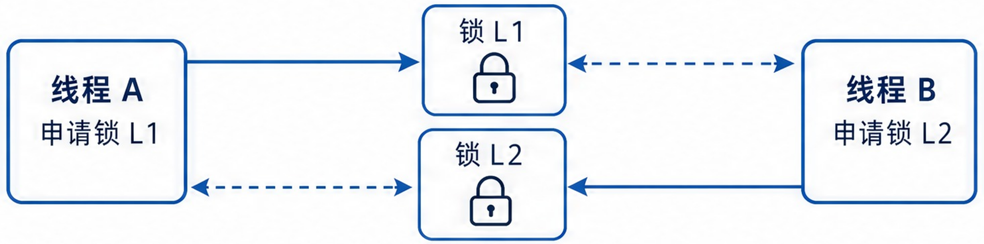
场景二： 输入框连续打字 → 流控防抖与迟到结果过滤



操作系统：看透虚拟化抽象背后的物理代价

抽象互斥模型

互斥锁与死锁环路模型

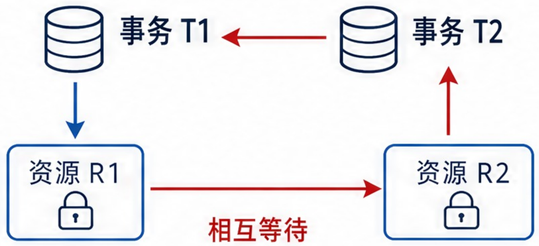


共享状态的排他访问与防死锁设计

业务超卖故障

秒杀抢购库存变负数与事务死锁排障案例

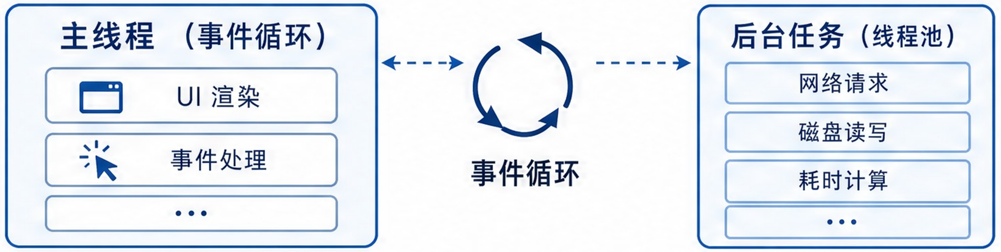
超卖导致库存为负数



事务死锁，业务阻塞

抽象调度模型

主线程事件循环与耗时任务隔离



主线程专注响应，耗时任务异步分流

业务阻塞故障

手机应用主线程同步读写导致界面冻结案例

阻塞主线程，体验差，可能被系统强退

核心启发： 高级语言创造了无限且独占的假象，但系统稳定性与性能始终受制于物理硬件规律。

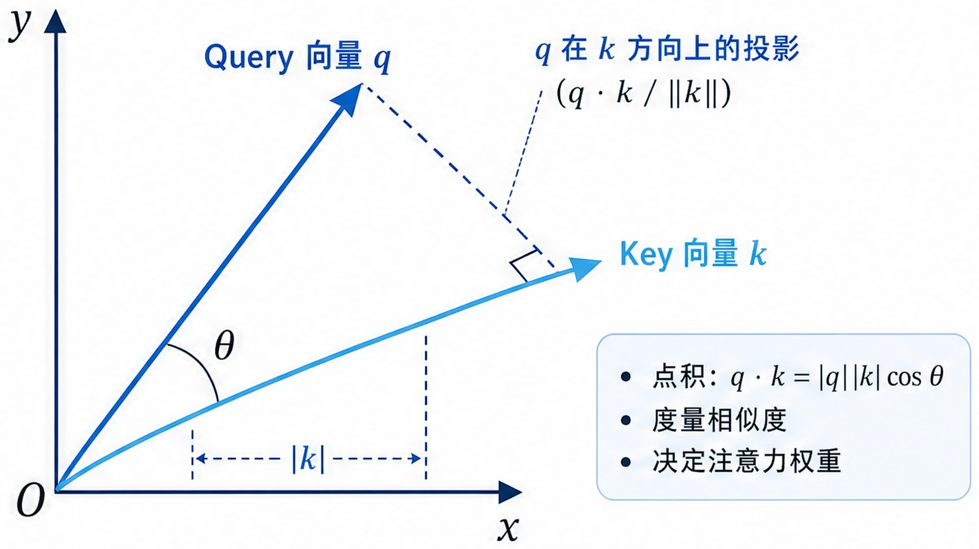
线性代数：高维几何空间与特征投影

从繁琐计算到几何直觉： 线性代数不是做矩阵算数，而是高维信息的空间几何与坐标变换

大模型的母语： Embedding 向量是高维空间中的坐标点，Attention 机制是 Query 与 Key 的点积投影与加权组合

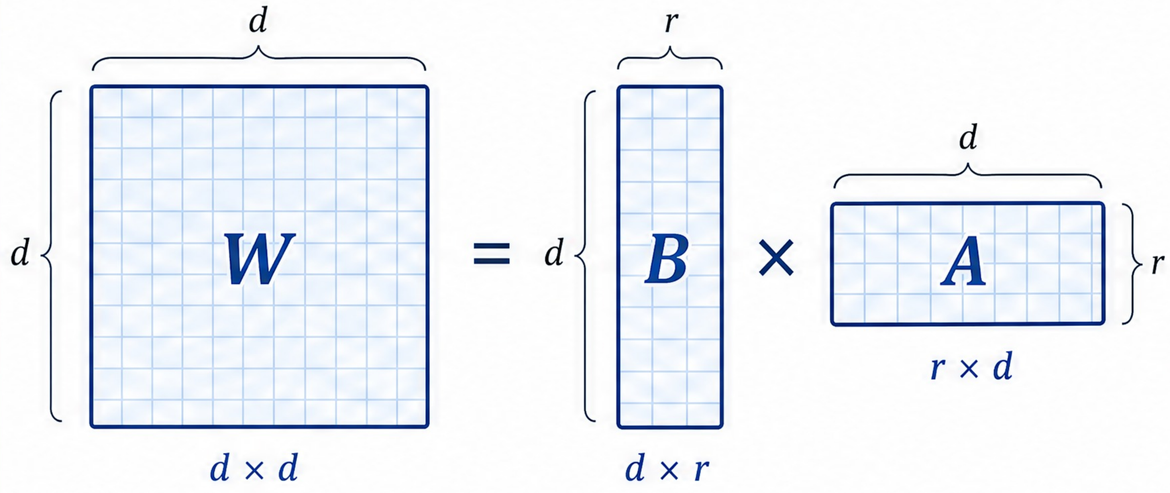
微调技术的数学底座： 大模型参数高效微调 LoRA（低秩自适应），本质是利用权重更新矩阵的内在低秩性进行数学分解（$W = B \times A$）

点积投影：Query 与 Key 的几何关系



- 点积：$q \cdot k = |q||k| \cos\theta$
- 度量相似度
- 决定注意力权重

LoRA 低秩分解：内在维度降维



LoRA 低秩分解：内在维度降维

$r \ll d$，显著减少可训练参数与计算成本

科班底座：看透未知系统的思维框架

超越机械背诵：

重要的不是死记协议报文、
系统调用或模型层级，
这些细节随时可以查阅

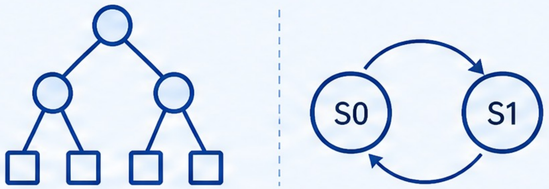
关注设计因果：

理解每个机制当初为了解决
什么矛盾而诞生，建立对
系统边界与物理代价的直觉

面对未知的迁移力：

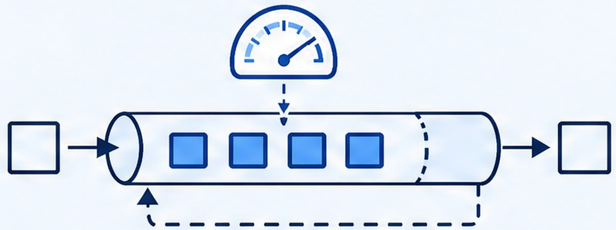
遇到全新的复杂系统时，
能够举一反三，认出其背后
的形式结构、通信容错与
资源瓶颈

编译原理



结构化秩序与状态机

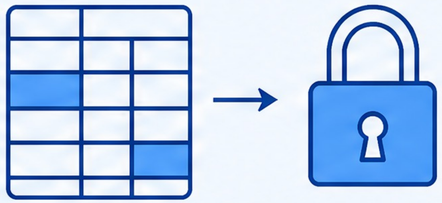
计算机网络



容量错配与反压流控

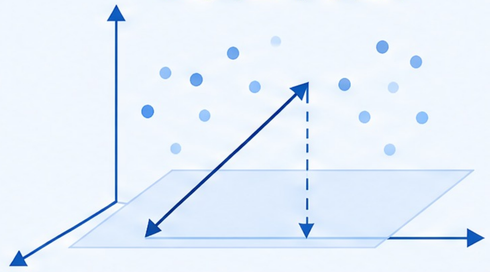
举一反三
与面对未知系统
的迁移力

操作系统



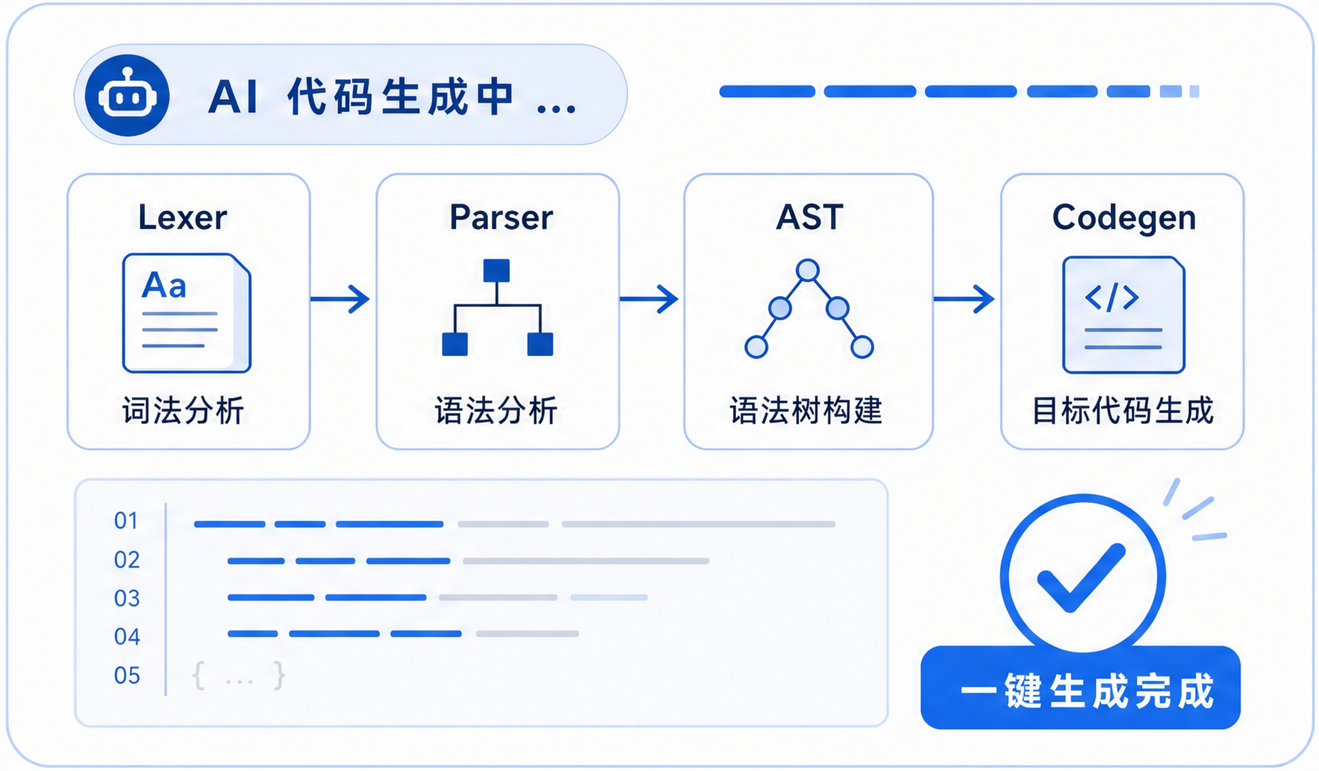
物理资源边界与并发状态

线性代数



数据几何与低秩分解

既然 AI 都能写大实验，我们为什么还要学？



?

代码跑通了，等于我们能解决问题吗？

看得懂
理解别人的代码

≠

做得出
解决真实的问题

现实的冲击：
大模型几秒钟就能生成词法分析、语法树构建与目标代码生成，甚至配齐测试脚本

核心追问：
代码跑通了，
等于我们能解决问题吗？

懂得的假象：
看别人写代码行云流水，自己动手才发现处处是火候与细节；能交出一份满分代码，不等于真正掌握了系统的脾气

这些底层思维，到底是越来越值钱了，还是越来越不需要了？

语法打字的贬值

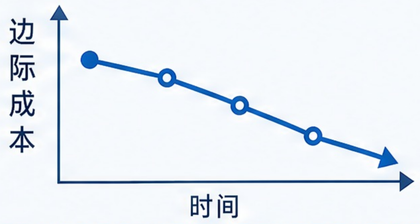
代码编写的翻译成本被大幅抹平，
单纯熟练敲语法的打字员正在变得不值钱

需求描述
自然语言

语法翻译
AI 自动生成

机械编写
复制粘贴 / 微调

代码产出
低门槛 / 可替代



边际成本趋零
逐渐不值钱

VS

底层直觉的升值

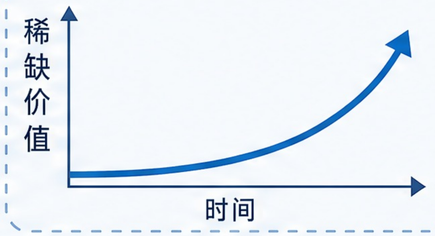
识别问题本质、定义系统契约、
选择用哪套思维模型去兜底，反而成了最稀缺的放大器

识别问题本质
抓住核心矛盾

定义系统契约
明确边界与规则

选择思维模型
模型匹配与抽象

构建系统底座
稳健可扩展



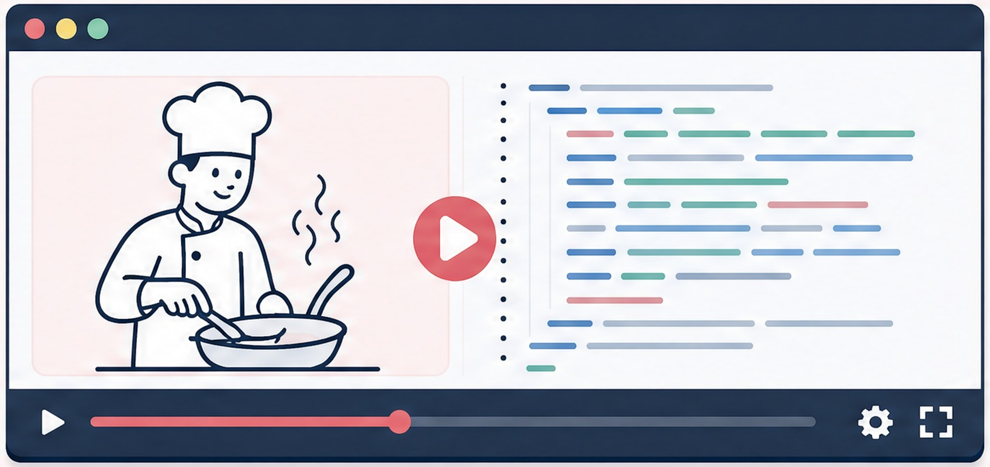
稀缺性骤增
成为放大器的
杠杆核心

放大器的逻辑

AI 是强大的动力引擎，
但你的系统底座越扎实，它被你放大的倍数才越高。

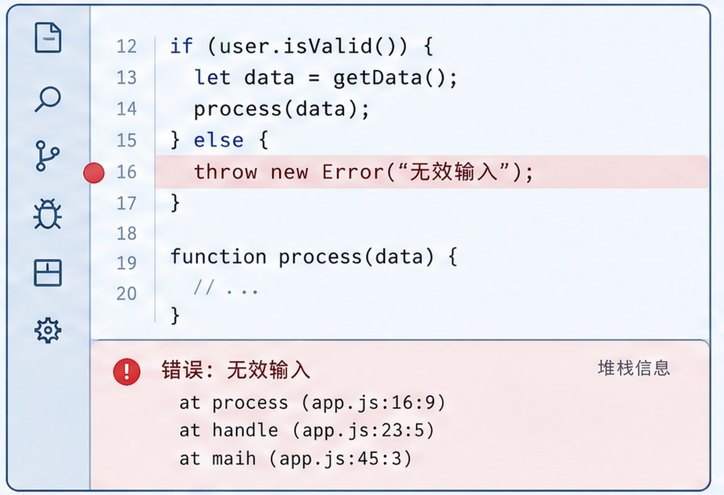
大脑：我会了；手：不，你没有——手感只能靠动手攒

观众视角（看懂的假象）



大脑疯狂分泌多巴胺，
产生"我已经会了"的幻觉

选手视角（动手的阻力）



系上围裙进厨房，
手立刻诚实表明：根本不会

手感只能靠动手去攒，不能靠看和背—— 由可行性、难度、决策三层判断力组成。

可行性判断
判断方向行不行、方案能不能跑通

难度判断
评估复杂度与成本，找到突破口

决策判断
权衡取舍，选择最优行动路径

三层判断力·一：可行性判断 ——这问题能不能系统化解决？边界在哪？

看到一个看似模糊的问题，判断它有没有形式结构、能不能用程序确定性解决；反过来，更看清它哪里解不了、边界卡在哪

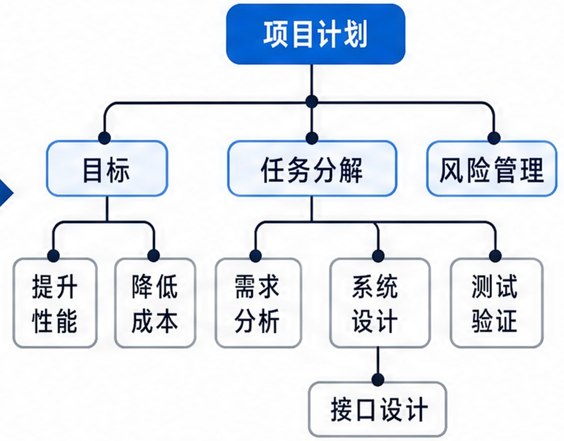
正例·认出确定性结构（缩进大纲）

一眼认出形式化 DSL，用简单程序确定性解析

缩进大纲文本

```
1  项目计划
2    ├─ 目标
3    │    ├─ 提升性能
4    │    └─ 降低成本
5    ├─ 任务分解
6    │    ├─ 需求分析
7    │    ├─ 系统设计
8    │    │    └─ 接口设计
9    │    └─ 测试验证
10   └─ 风险管理
```

映射为 AST（抽象语法树）



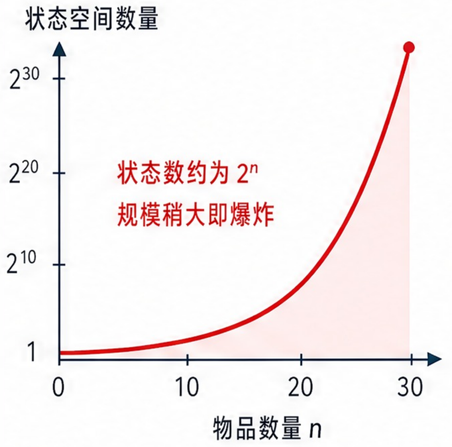
背后有文法规则， 结构明确， 可用递归下降等确定性方法解析

可行性边界划分

反例·看清不可行边界（0/1 背包）

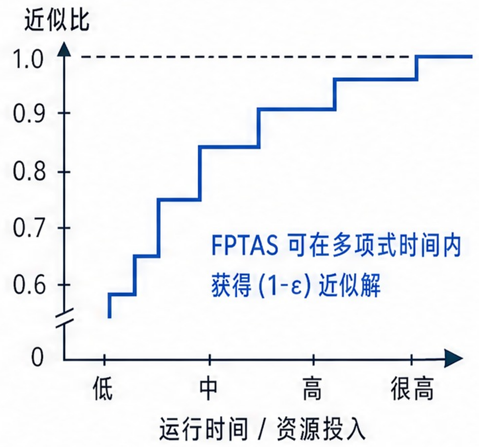
精确解在大规模下失效，看清边界果断转向近似算法，避免盲目空耗

精确解的复杂度：指数爆炸



时间与空间指数增长，
大实例下无法在合理时间内求得精确最优解

近似解的思路：可控近似比



牺牲少量精度换取可扩展性，
工程上可落地、可控、可迭代

可行性判断 = 认得出结构，更看清解的边界。

心里有那条边界：什么解得了、什么解不了、边界在哪 ——避免在一个做不成的方向空耗一个月。

三层判断力·二：难度判断——这事多大代价？难点真在哪？

核心定义：看到一个问题，能估出成本量级（半天还是半年），且看清难点不在表面算法，而在输入域、边界条件与工程集成

量级对比·成本差万倍

个人 DSL 解析器
- 输入域极小
- 规则简单可控

成本：半天

成本取决于输入域
与边界复杂度

生产级 C 编译器
- 语言规范庞大
- 无数 Corner Case

成本：十年

代价差万倍，源于输入域是自定义简单规则
还是整门语言规范与数十年 corner cases

结构降本·LoRA 数学杠杆

权重更新矩阵

$$\Delta W = B_{(r \times d_{out})} \times A_{(d_{in} \times r)}$$

内在低秩性：$r \ll d_{in}, d_{out}$

算力成本对比

GPU 集群训练

单卡微调

认出低秩结构，成本下降一个量级

底层直觉：难度判断 = 估得出量级，说得清成本住在哪一层，懂得利用结构杠杆化解复杂度

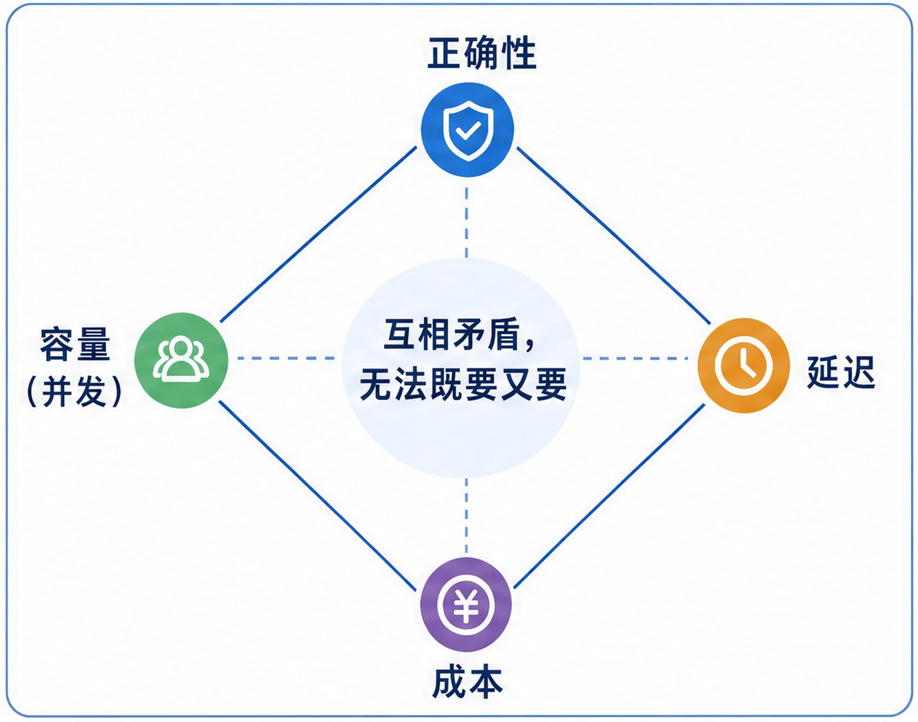
三层判断力·三：决策判断（高维权衡）—— 约束互相打架，在哪下刀？

核心定义

高维权衡是多个互相冲突的变量拉扯，没有教科书标准解，靠一手手感确定在哪下刀牺牲谁

不可能四角

正确性 × 容量（并发）× 延迟 × 成本，四者天然矛盾，绝无既要又要



秒杀库存三解法

1 降延迟提吞吐

目标	降低延迟，提升吞吐
牺牲项	牺牲强正确性 （可能暂时超卖）
具体解法	内存预扣 + 异步落库 （最终一致）

2 保绝对正确性

目标	保证绝不超卖
牺牲项	牺牲吞吐与延迟 （串行等待）
具体解法	数据库悲观锁 串行扣减（绝不超卖）

3 提容量控成本

目标	提升容量，控制成本
牺牲项	牺牲响应延迟 （需要排队等待）
具体解法	消息队列削峰 + 幂等去重（排队处理）

为什么必须靠手感

- 几十个维度互相打架
- 书本无法穷举
- 技巧只解局部
- 最佳实践等于泄露实现
- 唯有亲自动手趟过

怎么判断我们真的会了？

评测困境

代码大模型几秒跑通测试、生成漂亮报告，
表面看似完美，但无法分辨是真懂还是包装

测试结果	
单元测试	通过
集成测试	通过
性能测试	通过
安全扫描	通过

表面全绿，
能否分辨
真会还是包装？

评估设计的两条路线（类比模型训练）

路线一 End-to-End 全功能交付（大作业）

AI 助手 + 开发工具链 + 网络检索 → 项目交付
最终结果

优点：
贴近真实工程

痛点：
容易表面过拟合，
难以区分真会

路线二 Ablation 控制变量测试（闭卷）

AI 助手　开发工具链　网络检索 → 独立思考
从零判断

核心思路： 拿走所有外部辅助，单测大脑从零产生判断的基线

优势： 可扩展、可复制、能稳健区分真实能力边界

核心启发：闭卷考试是从评估设计中推导出来的控制变量实验，而非凭空强加的死板规矩

闭卷 = ablation / 控制变量 / proxy metric

评估本质：闭卷考试是“判断力”这个核心目标唯一可扩展的 Proxy Metric（代理指标），
考的是技术直觉，不是死记硬背

双重指标互补（类比模型训练）

大作业（End-to-End 探索上限）

探索工具协作上限，追求全局最优方案
易过拟合表面表现

互为补足
杜绝过拟合

闭卷考试（Ablation 守住独立底线）

切断外部工具干扰，检验独立技术判断
易过拟合应试技巧

只有 End-to-End Loss 会过拟合表面；只有 Proxy Metric 会过拟合应试；
两者互补才形成稳健评估

考试设计三原则

原则 1 · 控制变量

切断外力，隔离单测独立判断；

原则 2 · 抓大放小

重架构解耦，轻死板像素语法；

原则 3 · 理解区分

区分度建立在概念理解
而非记忆背诵；

核心认知：闭卷不是为难人，而是为你打磨即便切断所有工具依然稳固的底层底气

度量不等于目标，但好的 Proxy 始终指向真正升值的底层判断力。

技能的外包与内化：教育的目标没变，但学习其实变难了

外包给工具的技能

语法拼写、样板代码、
环境配置与函数查阅，
这些机械记忆交给 AI

语法拼写记忆
快速检索与修复

库函数查阅
随查随用即得

样板代码生成
快速复用与拼装

特点：标准化、可复用、快速商品化

必须内化的内核

系统分层直觉、权衡取舍能力、
多维约束下的破局思路

系统分层直觉
结构化理解与抽象能力

权衡取舍能力
在多目标间做出最优判断

复杂系统排障思路
定位本质、构建解决路径

特点：难以复制、长期复利、深度价值

学习的新挑战

过去背熟语法肯敲代码就能当合格码农；
现在机械工作被接管，学习反而更烧脑了——
我们必须**直面抽象本质**。

职场真相：高级工程师不是把初级工作做得更熟练

晋升误区

以为代码写得更快、更少 bug，
就会自动从初级工程师变成资深工程师

现实启发

初级工程师负责的明确翻译
正是 AI 最擅长的，
唯有尽早建立高层架构直觉
才能打开空间

核心洞察： 能力跃迁的关键不在“更快更好做执行”，而在“更高维度做’选择与设计”。

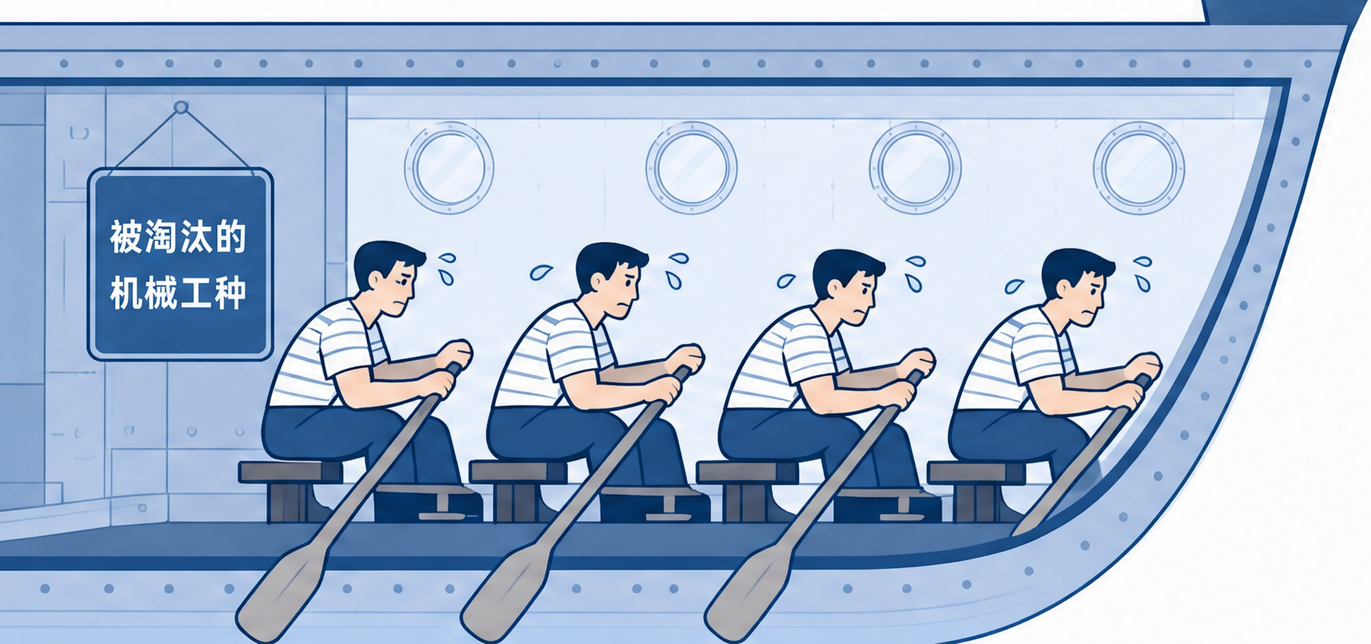
大航海时代：别再和 AI 比划桨了

划桨手的黄昏：

过去比拼数据结构肌肉记忆、代码规范和加班耐力，
就像比拼谁划桨更快

蒸汽机掀翻桌子：

AI 这台蒸汽机 24 小时不知疲倦，
人力划桨再快也比不过机器动力



走上甲板的新角色：

领航员： 判断商业与技术大方向，
决定这艘船究竟驶向哪里

轮机长： 深入系统内部，精细设定上下文、
约束与参数，驾驭强大的 AI 引擎稳定输出动力

从打字员走向为系统做决策和承担责任的人

执行与责任的解耦：

AI 可以瞬间生成十个看似合理的实现方案，
但无法替人类承担线上故障后果

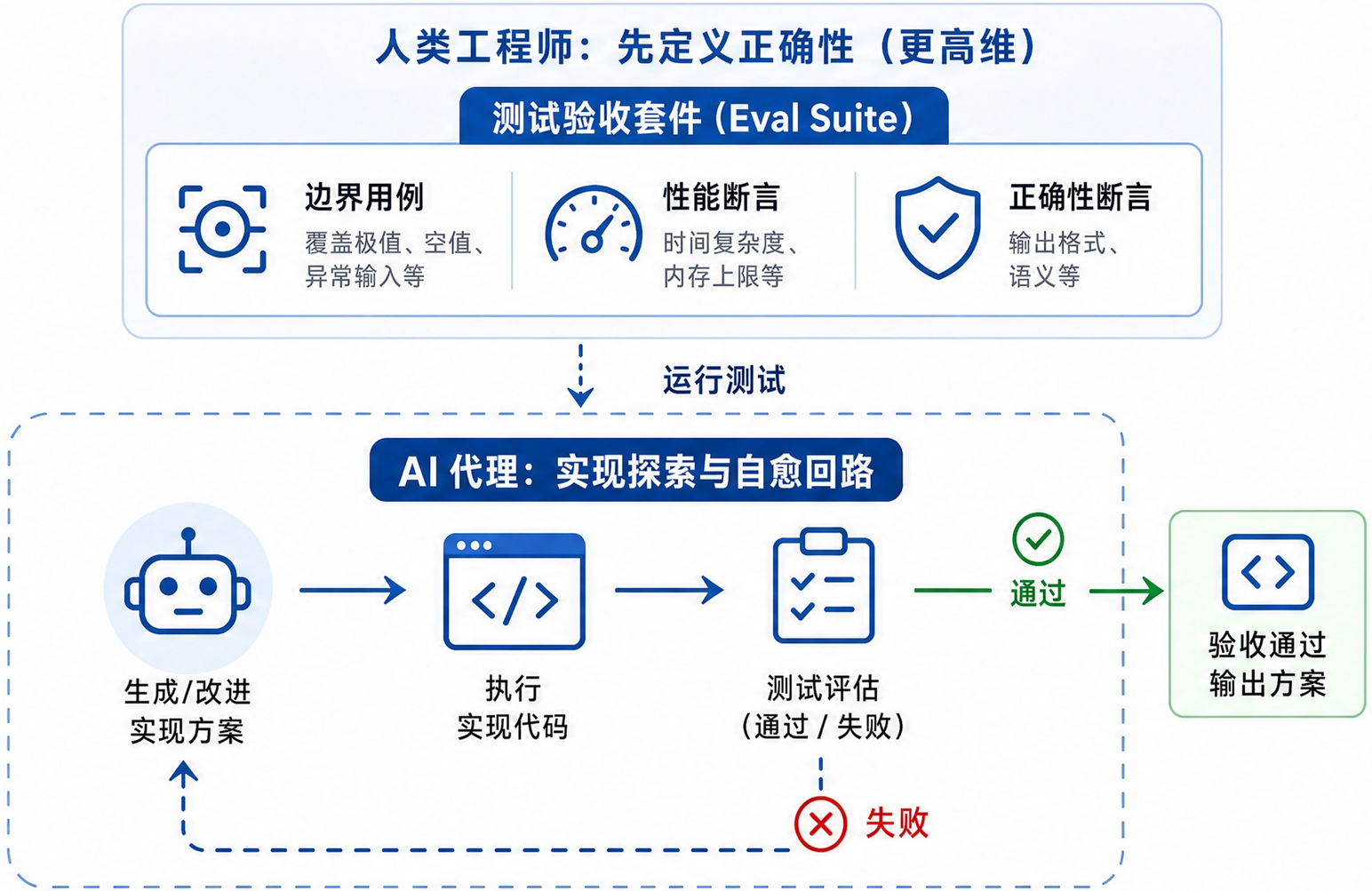
最终的责任人：

生产环境崩溃、数据损坏、架构走入死胡同，
最终被问责并解决问题的永远是人

核心护城河：

理解的深浅，决定了你能为多大复杂度的
系统兜底，决定了你在面对 AI 输出时敢不敢拍板

AI 辅助学习方法（一）：评估优先心智



1 先定义什么是正确

在让 AI 写任何代码之前，先逼自己把验收标准想清楚

2 测试驱动理解

设计边界测试、性能断言与异常输入；能写出一套严格验证 AI 的测试集，说明你真正理解了系统的边界

3 结果确定性的闭环

用明确的验收标准兜底，放手让 AI 探索多条实现路径并自动修正

定义正确性比单纯写实现更高维：高质量的问题 + 严格的评估，驱动可靠的结果。

AI 辅助学习方法（二）： 解决动机困惑

破除枯燥与虚无

不知道学这门课有什么用、
不知道和未来目标有什么关系时，
直接向 AI 咨询

个性化的行业映射

输入你的具体兴趣（比如量化、游戏、自动驾驶），让它梳理课程知识在这些领域的真实应用

保持独立判断

将其作为拓宽视野、打破信息差的启发工具，吸取息路，
而非盲目信奉每个结论

你的提问

我想做量化交易系统，
编译原理这门课对我有用吗？

AI 向导

非常有用！编译原理中的文法解析、词法分析、中间表示等技术，广泛应用于低延迟报文协议处理与交易系统的基础设施中。

核心映射关系

文法与解析器	→	报文协议解析器（如 FIX、ITCH、SBE）
词法分析	→	报文分隔与字段提取（高吞吐、零拷贝）
抽象语法树（AST）	→	报文结构化表示与规则校验
中间表示与优化	→	指令编排与策略脚本优化执行路径
代码生成与执行	→	低延迟执行引擎（微秒级响应）

在量化交易系统中的应用链路

市场数据接入
（交易所报文）

↓

协议解析器
（编译原理技术）

↓

结构化数据
与校验

↓

策略引擎
（量化模型）

↓

下单与风控
（低延迟执行）

打破专业信息差
激发底层学习动力

- 看见知识的真实价值
- 理解技术的底层逻辑
- 建立长期的学习动机

AI 辅助学习方法（三）：探讨设计权衡

转变交流方式

不直接索要代码答案，
而是深入请教设计思路
与底层原因

探讨替代方案

为什么这里选递归下降
而不是状态转移表？
如果换一种做法会有
什么开销？

挖掘适用条件

探寻每种经典算法和
架构在什么场景下成立、
在什么极限条件下会失效

设计问题

如何实现表达式语法解析？

需要在可读性、性能与工程成本之间做出权衡。

方案 A：递归下降解析

- 按文法结构编写函数
- 结构直观，易于调试
- 适合 LL(1) 及扩展文法

空间开销	★★☆	中等
时间复杂度	★★★☆	线性
实现阻力	★★★☆	较低

方案 B：状态转移表（LR）

- 自动化生成解析表
- 覆盖更广文法（LR(k)）
- 表大，调试与维护成本高

空间开销	★★★★	较高
时间复杂度	★★★☆	线性
实现阻力	★★★☆	较高

追问设计动机

为什么这样设计？
核心目标是什么？

对比替代方案

多方案横向比较，
权衡利弊得失。

探寻适用条件

什么场景下最合适？
何时会失效？

理性权衡

在不确定中做出
更优的工程决策。

AI 辅助学习方法（四）：动手造出来

新领域没有现成教材

前沿技术日新月异，书本往往滞后，
最好的学习方式就是动手做个最小完项目

模拟答辩检验真知

做完项目后，让 AI 扮演严苛的面试官，
就系统瓶颈、并发安全与设计选择向你提问

同伴交叉代码评审

同学之间互查代码、互相答辩，
能清晰解释每个关键权衡才算真正学会

① 动手造一个最小完整项目

项目脚手架

- 目录结构
- 构建配置
- 依赖管理

核心代码实现

- 核心功能
- 模块拆分
- 接口设计

测试与验证

- 单元测试
- 集成测试
- 运行验证

迭代改进
持续提升

② 模拟答辩与同伴互审

AI 扮演面试官（严苛提问）

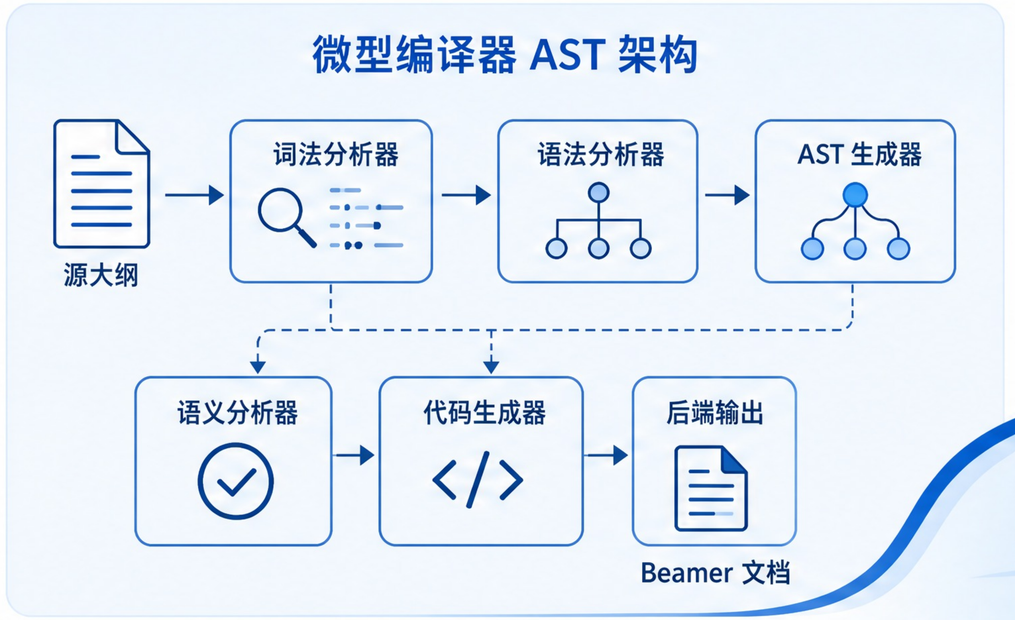
- 系统瓶颈在哪里？如何量化与优化？
- 并发安全如何保障？有哪些异常场景？
- 为何选择这种设计？有哪些权衡与替代？

同伴交叉代码评审

- 互查代码，指出问题
- 追问设计，质疑假设
- 解释权衡，达成共识

通过答辩才是真正掌握　能经得起提问与质疑，才算真正理解与掌握

回到微型编译器：大学里打磨出的眼光是一生的复利



小工具的启示

当年那个看似不起眼的大纲转 Beamer 编译器，浓缩了科班训练赋予我们的形式化抽象与解耦直觉

跨越周期的资产

编程语言、框架工具与命令参数会不断更迭，但看透系统结构、界定问题边界的判断力终身受用

拥抱新时代

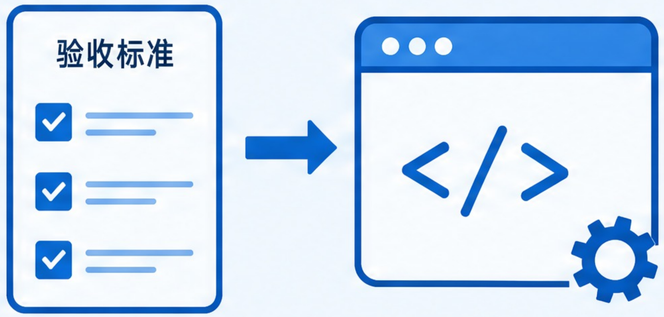
带着扎实的系统底座，坦然走出狭窄的划桨座位，走上甲板成为驾驭新时代引擎的人

给学弟学妹的一个行动建议：今晚找个真实小痛点，动手去 build

1 识别真实小痛点

在日常学业或生活中，找一个
让自己感到繁琐重复的小场景

2 设定评估标准与动手构建



先想清楚验收标准，借助 AI 快速搭建，
用真实的测试用例验证

3 记录认知收获与手感

记录下做之前觉得哪里难，做之后
发现真正难在哪里，迈出第一步

推荐工具链与讲义下载

Agent Harness

- dsh
- zcode
- workbuddy

LLM

- glm-5.3-flash
- qwen 3.8 flash
- qwen 3.8 27b

讲座幻灯片 PDF

ya@yage.ai

现场交流与问答